為什麼需要複數

實旋轉矩陣

$$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

當 $\theta \not\equiv 0 \pmod{\pi}$ 時，它沒有實特徵向量。特徵值為

$$\lambda = e^{\pm i\theta}.$$

複特徵向量讓振盪動態、單位根與週期訊號有自然表示。實矩陣的複特徵值成共軛對；實解由共軛特徵向量組合而成。

本章使用 $i^2 = -1$。電機工程常以 j 表示虛數單位，這只是記號慣例。

複數體的運算

$z = a + bi$，其中 $a, b \in \mathbb{R}$。加減法逐分量進行：

$$(a + bi) + (c + di) = (a + c) + (b + d)i.$$

乘法：

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i.$$

$\mathbb{C}$ 是一個體（field）。非零複數的乘法反元素為

$$\frac{1}{a + bi} = \frac{a - bi}{a^2 + b^2}.$$

複數乘法可交換，且非負的模使每個非零複數都有乘法反元素。$\mathbb{C}^n$ 可以直接使用標準基底，但係數是複數。

共軛、模與實維度

共軛為

$$\overline{a+bi} = a - bi.$$

模為

$$|z| = \sqrt{z\overline{z}} = \sqrt{a^2+b^2}.$$

共軛運算保持加法與乘法：

$$\overline{z+w} = \overline{z} + \overline{w}, \qquad \overline{zw} = \overline{z}\,\overline{w}.$$

$|zw| = |z||w|$，且三角不等式成立。

$\mathbb{C}^n$ 作為複向量空間的維度是 n；作為實向量空間的維度是 $2n$，因為每個座標都有實部與虛部。討論秩以前，必須先指明所用的體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極式與 Euler 公式

非零

$$z = a + bi$$

有

$$r = |z|, \qquad \theta = \operatorname{atan2}(b, a),$$
$$z = re^{i\theta} = r(\cos\theta + i\sin\theta).$$

Euler 公式為

$$e^{i\theta} = \cos\theta + i\sin\theta.$$

複數相乘時，模相乘而幅角相加：

$$(re^{i\theta})(se^{i\phi}) = rse^{i(\theta+\phi)}.$$

整數次冪為

$$(re^{i\theta})^k = r^k e^{ik\theta}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極式與 Euler 公式 （續）

例：

$$1 + i = \sqrt{2} e^{i\pi/4},$$
$$(1+i)^8 = 16 e^{2\pi i} = 16.$$

幅角加上 2π 仍表示同一個複數；分支的選擇會影響對數等多值函數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單位根

解

$$z^n = 1.$$

令

$$\omega = e^{2\pi i/n}.$$

所有單位根為

$$1, \omega, \omega^2, \dots, \omega^{n-1}.$$

它們等距落在單位圓上，形成正 n 邊形。

若 $z = re^{i\theta}$ 且 $z^n = 1$，則

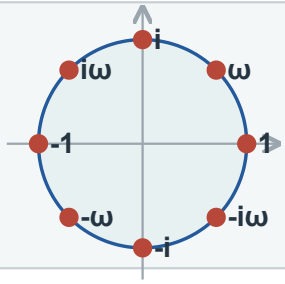
$$r = 1, \qquad n\theta = 2\pi k.$$

$n = 4$ 時，單位根是 $1, i, -1, -i$。這些根構成 Fourier 基底與循環排列特徵向量的核心。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單位根 (續)

八次單位根：$z^8=1$ 等距落在單位圓



$\omega=e^{(2\pi i/8)}$

roots：$1,\omega,\ldots,\omega^7$

$F_8F_8^H=8I$

columns 正交；Fourier basis

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複數內積

本章固定

$$\langle u,v\rangle = u^H v = \sum_j \overline{u_j} v_j,$$

其中

$$u^H = \overline{u^T}.$$

若 $u=(1,i)^T$：

$$\|u\|^2 = u^H u = |1|^2 + |i|^2 = 2,$$

不是 $1^2+i^2=0$。

內積具有以下性質：

$$\langle u,v\rangle = \overline{\langle v,u\rangle},$$
$$\langle u,v+w\rangle = \langle u,v\rangle + \langle u,w\rangle,$$
$$\langle u,\alpha v\rangle = \alpha\langle u,v\rangle,$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複數內積 （續）

$$\langle \alpha u, v \rangle = \overline{\alpha}\, \langle u, v \rangle.$$

本章把共軛固定在第一個引數。若改用其他慣例，所有公式必須同步調整。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正交性與四個基本子空間

$u \perp v$ 表示

$$u^H v = 0.$$

例：

$$u = (1, i)^T, \qquad v = (1, -i)^T,$$
$$u^H v = (1, -i)(1, -i)^T = 1 + (-i)(-i) = 1 - 1 = 0.$$

複數空間的四個基本子空間改用共軛轉置描述：

$$C(A) \perp N(A^H), \qquad C(A^H) \perp N(A).$$

一般轉置 A^T 不適用，因為它不保證給出複內積下的正交補空間。

維度關係為

$$\dim C(A) = \dim C(A^H) = r,$$
$$\dim N(A) = n - r, \qquad \dim N(A^H) = m - r.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hermitian 矩陣

方陣 S 是 Hermitian，若

$$S^H = S.$$

逐元素條件為

$$s_{ji} = \overline{s_{ij}}.$$

對角元素必須是實數。對任何 z，

$$(z^H S z)^H = z^H S^H z = z^H S z.$$

1×1 純量等於自己的共軛，故

$$z^H S z \in \mathbb{R}.$$

例：

$$S = \begin{bmatrix} 2 & 1+i \\ 1-i & 3 \end{bmatrix}$$

是 Hermitian。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hermitian 矩陣的實特徵值與正交特徵向量

若

$$Sz = \lambda z,$$

左乘 z^H：

$$z^H S z = \lambda z^H z.$$

左端是實數，且 $z^H z > 0$，所以 λ 是實數。

若

$$Su = \lambda u, \qquad Sv = \mu v,$$

則

$$\lambda u^H v = (Su)^H v = u^H S v = \mu u^H v.$$

當 $\lambda \neq \mu$，

$$(\lambda - \mu) u^H v = 0,$$

所以 $u \perp v$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hermitian 矩陣的實特徵值與正交特徵向量 （續）

完整正交特徵基底的證明與實對稱情形相同：在複單位球上最大化實值 $z^H S z$，由實部、虛部方向的極值條件得到 $Sq = \lambda q$。$q^\perp$ 因 $q^H S v = (Sq)^H v = 0$ 而不變；限制到 $q^\perp$ 的映射仍為 Hermitian，歸納可得完整基底。各重複特徵空間內部再用 Gram-Schmidt，即得譜定理：

$$S = Q \Lambda Q^H,$$

其中 $Q^H Q = I$，且 Λ 為實對角矩陣。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unitary 矩陣

方陣 $Q \in \mathbb{C}^{n \times n}$ 稱為 unitary，若

$$Q^H Q = I.$$

方陣亦滿足 $QQ^H = I$，所以

$$Q^{-1} = Q^H.$$

它保持內積：

$$\langle Qu, Qv \rangle = (Qu)^H (Qv) = u^H v.$$

因此 $\|Qu\| = \|u\|$。

若 $Qu = \lambda u$：

$$\|u\| = \|Qu\| = |\lambda| \|u\|,$$

所以

$$|\lambda| = 1.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unitary 矩陣 （續）

行列式也直接反映長度保持：

$$1 = \det(Q^H Q) = (\det Q)^H \det Q = |\det Q|^2,$$

故 $|\det Q| = 1$。實正交矩陣是 unitary 矩陣的實數特例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正規矩陣

方陣 A 稱為正規矩陣（normal matrix），若

$$AA^H = A^H A.$$

複方陣可用 unitary 矩陣對角化，若且唯若它是正規矩陣。充分性可用 Schur 形式歸納證明：先取單位特徵向量並擴充為正交歸一基底，矩陣變成 $\begin{bmatrix} \lambda & b^H \\ 0 & C \end{bmatrix}$。正規條件比較 $(1,1)$ 元素，給出 $|\lambda|^2 + \|b\|^2 = |\lambda|^2$，故 $b = 0$；C 仍為正規矩陣，歸納即得 unitary 對角化。必要性可直接由 $A = Q\Lambda Q^H$ 左右相乘驗證。三個常見類別為

- Hermitian：特徵值為實數；
- 反 Hermitian 矩陣 $K^H = -K$：特徵值為純虛數；
- unitary：特徵值的模為 1。

一般可對角化只要求特徵向量基底；unitary 可對角化則要求正交歸一特徵基底。後者是譜幾何中更強的條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hermitian 對角化例子

$$S = \begin{bmatrix} 2 & i \\ -i & 2 \end{bmatrix} = S^H.$$

特徵多項式為

$$\det(S - \lambda I) = (2-\lambda)^2 - |i|^2 = (\lambda - 1)(\lambda - 3).$$

特徵值為 $1, 3$。

對 $\lambda = 3$：

$$\begin{bmatrix} -1 & i \\ -i & -1 \end{bmatrix} z = 0$$

給 $z_1 = iz_2$，取

$$q_1 = (i, 1)^T / \sqrt{2}.$$

對 $\lambda = 1$，取

$$q_2 = (-i, 1)^T / \sqrt{2}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hermitian 對角化例子 （續）

兩個向量都是單位向量，且 $q_1^H q_2 = 0$。因此

$$S = Q \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} Q^H,$$

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} i & -i \\ 1 & 1 \end{bmatrix}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fourier 矩陣

令

$$\omega = e^{2\pi i/n}.$$

正向 Fourier 矩陣為

$$(F_n)_{jk} = \omega^{jk}, \qquad 0 \le j, k < n.$$

正向轉換為

$$y = F_n c.$$

F_n 是對稱矩陣；當 $n > 2$ 時一般不為 Hermitian，只有 $n = 1, 2$ 時是實對稱矩陣。各 columns 彼此正交：

$$\sum_{j=0}^{n-1} (\omega^{jk})^H \omega^{j\ell} = \sum_{j} \omega^{j(\ell-k)} = n\delta_{k\ell}.$$

所以

$$F_n F_n^H = nI, \qquad F_n^{-1} = F_n^H / n.$$

反 DFT 與 Parseval 恆等式

反轉換為

$$c_j = \frac{1}{n} \sum_{k=0}^{n-1} \omega^{-jk} y_k.$$

若軟體定義

$$\omega = e^{-2\pi i/n},$$

這只表示軟體交換了正向與反向轉換的符號慣例。

Parseval：

$$\|y\|^2 = (F_n c)^H (F_n c) = n\|c\|^2.$$

正規化的 unitary Fourier 矩陣為

$$Q = F_n / \sqrt{n}$$

它滿足 $Q^H Q = I$，故 $\|Qc\| = \|c\|$。正規化不改變轉換所承載的資訊，但會改變 Parseval 公式中的常數。

DFT 與循環卷積

定義循環卷積 $h_j = \sum_{k=0}^{n-1} a_k b_{(j-k) \bmod n}$。它在 Fourier 域中變成逐點相乘：

$$F_n h = (F_n a) \odot (F_n b)$$

證明：將 h_j 代入 $\sum_j \omega^{\ell j} h_j$，令 $s = (j-k) \bmod n$，雙重和分離成 $(\sum_k a_k \omega^{\ell k})(\sum_s b_s \omega^{\ell s})$。此公式使用未正規化 forward DFT；若 forward 除以 $\sqrt{n}$，右端需再乘 $\sqrt{n}$。

FFT、逐點相乘與逆 FFT 合計需 $O(n \log n)$，低於直接計算一般卷積的 $O(n^2)$。若兩序列長度分別為 p, q，線性卷積共有 $p + q - 1$ 項，必須補零到至少這個長度，才不會發生循環繞回。

若公式中的係數順序或符號與卷積定義不一致，結果會悄然改變，因此實作時應寫清楚 NTT／FFT 的慣例。

Radix-2 FFT 的奇偶分解

設 $n = 2m$、$0 \le j < m$。把序列分成

$$c' = (c_0, c_2, \dots, c_{2m-2}),$$
$$c'' = (c_1, c_3, \dots, c_{2m-1}).$$

將 $y_j = \sum_{k=0}^{n-1} c_k \omega_n^{jk}$ 分成偶數 $k = 2s$ 與奇數 $k = 2s+1$ 的兩個和。因 $\omega_n^2 = \omega_m$ 且 $\omega_n^m = -1$，

$$y_j = y'_j + \omega_n^j y''_j,$$
$$y_{j+m} = y'_j - \omega_n^j y''_j.$$

其中 y'、y'' 是兩個大小為 $m = n/2$ 的 DFT。

這就是蝶形（butterfly）運算：把大小為 n 的問題拆成兩個半尺寸子問題，再加上 m 次 twiddle 乘法。遞迴到大小為 1 即完成 FFT。

以 $n = 4$、$c = (1, 2, 0, -1)^T$ 逐步計算。位元反轉先得到

$$(c_0, c_2, c_1, c_3) = (1, 0, 2, -1).$$

Radix-2 FFT 的奇偶分解 （續）

第一層執行兩次兩點蝶形：

$$(1,0) \mapsto (1,1), \qquad (2,-1) \mapsto (1,3).$$

第二層的 twiddle 因數是 $\omega_4^0 = 1, \omega_4^1 = i$：

$$(1,1) \mapsto (2,0), \qquad (1,3) \mapsto (1+3i, 1-3i).$$

因此

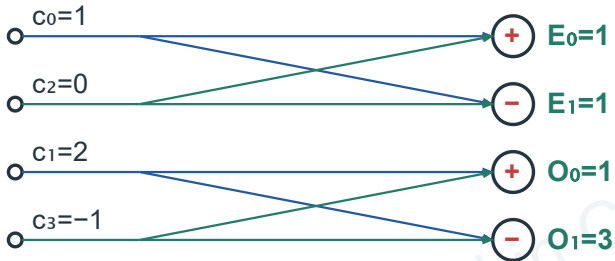
$$y = F_4 c = (2, 1+3i, 0, 1-3i)^T.$$

直接代公式亦得 $y_1 = 1 + 2i + (-1)(-i) = 1 + 3i$、$y_3 = 1 - 2i + (-1)i = 1 - 3i$。蝶形的加、減兩支共用同一個乘以 twiddle 的中間值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Radix-2 FFT 的奇偶分解 （續）

四點 FFT 第一層：分別計算偶序與奇序的兩點 DFT



偶序 $(c_0, c_2) = (1, 0)$

$E_0 = c_0 + c_2$，$E_1 = c_0 - c_2$

奇序 $(c_1, c_3) = (2, -1)$

$O_0 = c_1 + c_3$，$O_1 = c_1 - c_3$

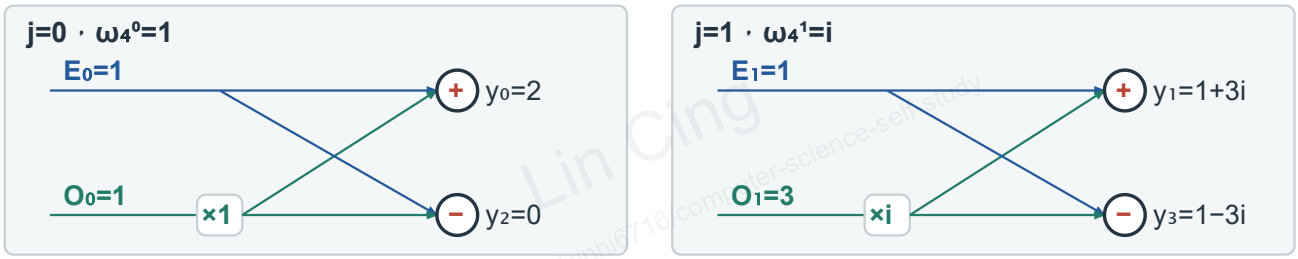
各蝶形的上支相加、下支以藍色值減去綠色值；下一圖再組合同索引 E_j、O_j。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四點 FFT 的重組：哪兩個中間值相配

第一層的陣列位置依序為 $(E_0, E_1, O_0, O_1) = (1, 1, 1, 3)$。第二層必須配對位置 $(0, 2)$ 與 $(1, 3)$：同索引的 E_j、O_j 才能組成 y_j 與 y_{j+2}。圖將兩個重組蝶形分開，以免交叉連線遮住配對關係。

四點 FFT 第二層：同索引的 E_j、O_j 分組重建 y



藍支 $u = E_j$，綠支 $\omega v = \omega_4^j O_j$：上支 $u + \omega v$，下支 $u - \omega v$；最後依 y_0, y_1, y_2, y_3 排列。

原例前三個中間值恰巧相等，會掩蓋錯接線。改用 $c = (2, 5, 7, -1)$，第一層為 $(9, -5, 4, 6)$，正確輸出為 $(13, -5 + 6i, 5, -5 - 6i)$。如果錯把 E_0, E_1 配在一起，第一個輸出會變成 $9 + (-5) = 4$，而不是 13。配對索引是演算法的一部分，不能只看一組巧合數值驗證。

遞迴與成本

$n = 2^\ell$：

$$T(n) = 2T(n/2) + \Theta(n).$$

共有 $\ell = \log_2 n$ 層，每層做線性工作量，因此

$$T(n) = \Theta(n \log n).$$

直接以 Fourier 矩陣相乘需要

$$n^2$$

次乘法。

$n = 1024 = 2^{10}$ 時，

$$n^2 = 1,048,576,$$
$$\frac{n}{2} \log_2 n = 5,120.$$

在此乘法成本模型下，兩者比值約為 205。實際執行時間仍受加法、快取、記憶體頻寬、平行化與 twiddle 因數生成影響。

迭代式 FFT 與位元反轉

遞迴 FFT 可改寫為由下而上的階段。$n = 8$ 時，radix-2 輸入通常先排成位元反轉順序：

$$0, 4, 2, 6, 1, 5, 3, 7.$$

再依序執行長度 $2, 4, 8$ 的蝶形：

$$(a, b) \mapsto (a + \omega b, a - \omega b).$$

反轉換可用共軛技巧：先對輸入取共軛、執行正向 FFT、對輸出取共軛，最後除以 n。

當 n 不是 2 的冪時，可使用混合基底（mixed radix）；結構不適合時可用 Bluestein 演算法。實輸入可使用實數 FFT 節省部分工作，但不能假設一般複數 API 會自動完成這些最佳化。

數值注意事項

DFT 的實務問題包括：

- 使用穩定的 twiddle 遞推或預先計算單位根；
- 不要累積大量相位誤差；
- 明確標示使用 F、$F/\sqrt{n}$ 以及反向轉換的符號；
- 補零會改變頻率格點與插值，不會創造新資訊；
- 窗函數能降低 leakage，但也會改變訊號；
- 頻率格點由 $k/(n\Delta t)$ 決定，取決於取樣間隔。

浮點頻譜峰值可能同時包含真實訊號、leakage 與雜訊。不能把一個非零頻格直接當成精確的物理頻率。

章末檢查

讀者應能：

- 計算複數乘積、模、極式冪與單位根；
- 固定 $u^H v$ 的共軛位置慣例，推導正交性；
- 證明 Hermitian 矩陣的特徵值為實數，且不同特徵空間互相正交；
- 建構 $S = Q\Lambda Q^H$；
- 驗證 unitary 矩陣保持長度，且特徵值的模為 1；
- 建 F_n，推導 $F_n F_n^H = nI$；
- 實作 radix-2 蝶形，計算 $\Theta(n \log n)$；
- 解讀符號、正規化、補零與數值極限。

下一章把這些結構應用到網路、工程、Markov 動態、最佳化、Fourier 級數、電腦圖學與密碼學。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Gilbert Strang. Introduction to Linear Algebra, 5th Edition, 2016.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study